



Maze Solving

- **Introduction**

- Consider the maze to be a black and white image, with black pixels representing walls, and white pixels representing a path. Two white pixels are special, one being the entry to the maze and another exit.
- Given such a maze, we want to find a path from entry to the exit.

- **Modelling the Maze**

- 0 -> Road
- 1 -> Wall
- 2 -> Maze entry
- 3 -> Maze exit
- 4 -> Cell part of the path from entry to exit

Source			
			Dest.

Gray blocks are dead ends (value = 0).

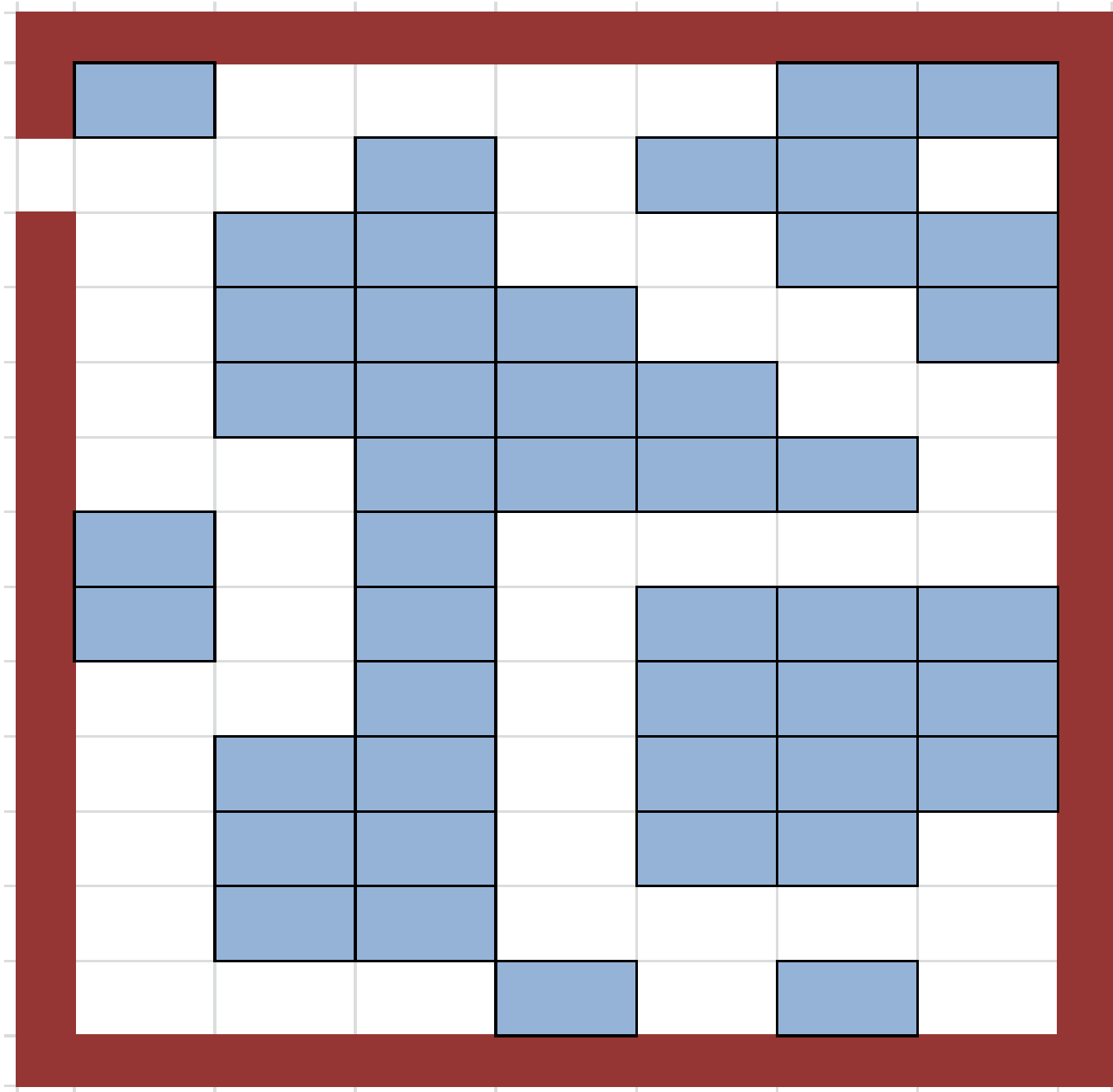
binary matrix representation of the above maze.

{ 1, 0, 0, 0 }

{1, 1, 0, 1 }

{0, 1, 0, 0 }

{1, 1, 1, 1 }



```

1. public class RatMaze {
2.     final int N = 4;
3.     void printSolution(int sol[][]){
4.         for (int i = 0; i < N; i++) {
5.             for (int j = 0; j < N; j++)
6.                 System.out.print(" " + sol[i][j] + " ");
7.             System.out.println();
8.         }}
9.     boolean isSafe(int maze[][], int x, int y){
10.         return (x >= 0 && x < N && y >= 0 && y < N
&& maze[x][y] == 1);
11.     }
12. {
13.     int sol[][] = { { 0, 0, 0, 0 },
14.                     { 0, 0, 0, 0 },
15.                     { 0, 0, 0, 0 },
16.                     { 0, 0, 0, 0 } };

```

```

1. if (solveMazeUtil(maze, 0, 0, sol) == false) {
2.     System.out.print("Solution doesn't exist");
3.     return false;
4. }
5.     printSolution(sol);
6.     return true;
7. }
8. boolean solveMazeUtil(int maze[][], int x, int y,int sol[][]){
9.     if (x == N - 1 && y == N - 1) {
10.         sol[x][y] = 1;
11.         return true;
12.     }
13. if (isSafe(maze, x, y) == true) {
14. sol[x][y] = 1;
15. if (solveMazeUtil(maze, x + 1, y, sol))
16.         return true;
17. if (solveMazeUtil(maze, x, y + 1, sol))
18.         return true;

```

```

sol[x][y] = 0;
    return false;
}
return false;
}

public static void main(String args[]){
    RatMaze rat = new RatMaze();
    int maze[][] = { { 1, 0, 0, 0 },
                     { 1, 1, 0, 1 },
                     { 0, 1, 0, 0 },
                     { 1, 1, 1, 1 } };
    rat.solveMaze(maze);
}
}

```

Output:

```

1 0 0 0
1 1 0 0
0 1 0 0
0 1 1 1

```

```

maze[][]={ {1, 1, 1, 1, 1},
            {0, 0, 1, 0, 1},
            {1, 1, 1, 1, 0},
            {1, 0, 0, 0, 1},
            {1, 1, 1, 1, 1}
};

```



THANK YOU